Aprendizaje por Refuerzo

Práctica 1

Universidad de Murcia
Máster en Inteligencia Artificial

Autores:
Montesinos Bravo, Jorge
j.montesinosbravo@um.es

Oliva Marín, Manuel
manuel.olivam@um.es

Ouardi Bennane, Mariam
mariam.ouardib@um.es

Profesor:
Hernández Molinero, Luis Daniel

Asignatura:
Extensiones de Machine Learning

Curso académico:
2025-2026

Fecha:
12 de marzo de 2026

Resumen

Este trabajo presenta un estudio exhaustivo sobre el aprendizaje por refuerzo, estructurado en dos grandes bloques que recorren desde los fundamentos teóricos hasta la implementación de técnicas avanzadas para el control de agentes en entornos complejos. El primer bloque se centra en el problema del bandido de k-brazos, un escenario fundamental para analizar el equilibrio entre la exploración de nuevas acciones y la explotación de la información conocida. A través de experimentos con distribuciones de recompensa normales, binomiales y de Bernoulli, se evalúa el rendimiento de algoritmos clásicos como ϵ-Greedy, UCB1 y Softmax. Los resultados confirman que estrategias basadas en la incertidumbre, como UCB1, ofrecen una convergencia superior al optimizar la reducción del arrepentimiento acumulado sin necesidad de ajustes manuales constantes.

El segundo bloque aborda el aprendizaje en entornos complejos, dividiéndose en métodos tabulares y control mediante aproximación de funciones. En el ámbito tabular, se comparan algoritmos de aprendizaje por Diferencia Temporal (SARSA y Q-Learning) frente a métodos de Monte Carlo en el entorno discreto *Cliff Walking* [6]. Se observa que los métodos de Diferencia Temporal, especialmente Q-Learning por su naturaleza *off-policy*, permiten una convergencia más rápida hacia políticas óptimas, mientras que SARSA tiende a aprender comportamientos más conservadores al ser un método *on-policy*.

Finalmente, se explora el control en espacios de estados continuos mediante el entorno *Mountain Car* [7]. Dado que el entorno original presenta un espacio de estados continuo, se realiza una discretización mediante agregación de estados utilizando una partición de 20×40 *bins*, lo que permite trabajar con una representación finita del estado. Sobre esta discretización se aplican distintos métodos de control con aproximación de función, incluyendo SARSA semi-gradiente con aproximadores lineales y *Deep Q-Networks* (DQN) basados en redes neuronales profundas. El análisis demuestra que la incorporación de redes neuronales, junto con técnicas de estabilidad como la memoria de repetición de experiencias y las redes objetivo, permite al agente generalizar mejor el conocimiento y mejorar el rendimiento. En conjunto, la práctica evidencia la evolución necesaria desde soluciones tabulares hacia aproximaciones paramétricas para abordar desafíos de mayor dimensionalidad en el aprendizaje por refuerzo moderno.

Toda la experimentación y el código asociado se encuentran disponibles en un repositorio de GitHub [5].

Índice general

1. Problema del Bandido de k-brazos **2**

1.1. Introducción . . . 2

1.2. Desarrollo . . . 2

1.3. Algoritmos . . . 2

1.3.1. Tipos de brazos . . . 3

1.4. Experimentos . . . 3

1.4.1. Distribución normal . . . 4

1.4.2. Distribución binomial . . . 4

1.4.3. Distribución bernoulli . . . 5

1.5. Conclusiones . . . 6

2. Aprendizaje en Entornos Complejos **7**

2.1. Introducción . . . 7

2.2. Métodos tabulares . . . 7

2.2.1. Desarrollo . . . 7

2.2.2. Algoritmos . . . 7

2.2.3. Experimentos . . . 9

2.2.4. Conclusiones . . . 10

2.3. Control con aproximaciones . . . 11

2.3.1. Desarrollo . . . 11

2.3.2. Algoritmos . . . 11

2.3.3. Experimentos . . . 12

2.3.4. Conclusiones . . . 14

2.4. Conclusiones generales . . . 14

Capítulo 1

Problema del Bandido de k-brazos

1.1. Introducción

El aprendizaje por refuerzo estudia cómo un agente interactúa con su entorno para tomar decisiones óptimas, abordando dilemas como el del bandido de k-brazos y aplicándose a tareas que van desde la optimización financiera hasta modelos de lenguaje avanzados.

La motivación de este estudio es comprender y aplicar los fundamentos del aprendizaje por refuerzo en escenarios prácticos, evaluando cómo diferentes algoritmos equilibran la obtención de recompensas inmediatas con la exploración del entorno mediante métricas como el *regret*.

El objetivo del presente informe consiste en diseñar, implementar y evaluar actividades centradas en conceptos fundamentales de aprendizaje por refuerzo, resolviendo problemas, comparando algoritmos y documentando los resultados de forma clara.

El informe se estructura en secciones que incluyen desarrollo teórico, definición formal de los problemas, descripción de algoritmos en pseudocódigo, evaluación experimental con gráficos, y un resumen de resultados y posibles líneas futuras de trabajo.

1.2. Desarrollo

El trabajo se centra en el problema del bandido de k-brazos, un caso simplificado de aprendizaje por refuerzo donde un agente elige repetidamente entre k acciones, cada una generando recompensas de una distribución estacionaria. El objetivo es maximizar la recompensa acumulada a lo largo de un horizonte temporal, sin cambios de estado en el entorno, lo que permite focalizarse en la evaluación de las acciones y la gestión de la incertidumbre.

En la literatura se han explorado diversos enfoques para equilibrar exploración y explotación. Estrategias clásicas como ϵ-Greedy [1] seleccionan la acción óptima la mayor parte del tiempo, reservando una fracción para explorar aleatoriamente, mientras que métodos como UCB1 [2] emplean límites de confianza superiores para guiar la exploración hacia acciones menos visitadas, optimizando la reducción del *regret* acumulado.

Para comprender la dinámica de estos algoritmos, se asume que el agente mantiene estimaciones tabulares de valor $Q_t(a)$, actualizadas mediante promedios incrementales. Esto permite observar cómo la tasa de exploración y otros parámetros afectan a la convergencia y la estabilidad de las estimaciones, proporcionando una base conceptual para métodos más avanzados como el aprendizaje por refuerzo aproximado.

En este capítulo se implementan y comparan tres estrategias de selección de acciones: ϵ-Greedy, UCB1 y Softmax. El análisis experimental se centra en cómo las propiedades del entorno influyen en su rendimiento, evaluando la exploración y explotación mediante experimentos controlados que permiten comparar la estabilidad y eficiencia de cada algoritmo bajo distintos niveles de variabilidad en las recompensas.

1.3. Algoritmos

Para el desarrollo experimental se seleccionaron tres algoritmos fundamentales del aprendizaje por refuerzo que ilustran enfoques distintos para resolver el dilema entre exploración y explotación, siguiendo la clasificación de Sutton y Barto [1]: estrategias básicas (aleatorias), optimistas (basadas en incertidumbre) y probabilísticas (basadas en valor).

Algorithm 1 Algoritmo ϵ-Greedy Incremental para Bandido de k-brazos

Inicializar: para cada acción $a = 1, \ldots, k$:
  $Q(a) \leftarrow 0$
  $N(a) \leftarrow 0$
Repetir para cada paso t:
if $\epsilon = 0$ y existe algún brazo a con $N(a) = 0$ **then**
  Seleccionar al azar un brazo A entre los no probados
else
  Generar un número aleatorio $p \in [0, 1]$
  if $p < \epsilon$ **then**
    $A \leftarrow$ acción aleatoria (exploración)
  else
    $A \leftarrow \operatorname{argmax}_a Q(a)$ (explotación)
  end if
end if
Ejecutar acción A y obtener recompensa R
$N(A) \leftarrow N(A) + 1$
$Q(A) \leftarrow Q(A) + \frac{1}{N(A)}(R - Q(A))$

El primero es el algoritmo ϵ-Greedy. Su comportamiento es principalmente codicioso, seleccionando la acción con mayor valor estimado, pero con probabilidad ϵ explora aleatoriamente. Las estimaciones $Q(a)$ se actualizan incrementalmente tras cada acción, y para $\epsilon = 0$ se asegura un recorrido inicial de todos los brazos, garantizando que cada acción sea probada

al menos una vez. A pesar de su simplicidad, ϵ-Greedy es efectivo en entornos estacionarios, aunque no discrimina entre acciones casi óptimas y las peores durante la exploración. El procedimiento se detalla en el Algoritmo 1.

Algorithm 2 Algoritmo UCB1

Inicializar: Ejecutar cada acción a una vez para evitar división por cero.
Actualizar $Q(a)$ y fijar $N(a) = 1$ para todo a.
$t \leftarrow k$
Repetir indefinidamente:
  $t \leftarrow t + 1$
  Seleccionar acción A que maximice:
  $A \leftarrow \underset{a}{\operatorname{argmax}} \left[Q(a) + c\sqrt{\frac{\ln t}{N(a)}} \right]$
  Obtener recompensa R
  $N(A) \leftarrow N(A) + 1$
  $Q(A) \leftarrow Q(A) + \frac{1}{N(A)}(R - Q(A))$

El segundo algoritmo es UCB1 (*Upper Confidence Bound*), un método determinista basado en el principio de optimismo ante la incertidumbre. Cada acción se prueba inicialmente una vez, y luego se selecciona aquella que maximiza un límite superior de confianza, lo que fuerza la exploración de brazos poco visitados y reduce la incertidumbre sobre su valor estimado. Esta estrategia evita la aleatoriedad y optimiza la transición entre exploración y explotación, como se muestra en el Algoritmo 2.

Algorithm 3 Algoritmo Softmax

Inicializar: $Q(a) \leftarrow 0$, $N(a) \leftarrow 0$ para todo a.
Parámetro: Temperatura $\tau > 0$
*Inicialmente se seleccionan los brazos que no han sido explorados
Repetir para cada paso t:
  Calcular probabilidades de selección $\pi_t(a)$:
  $\pi_t(a) = \frac{e^{Q(a)/\tau}}{\sum_{b=1}^{k} e^{Q(b)/\tau}}$
  Seleccionar acción A acorde a la distribución π_t
  Obtener recompensa R
  $N(A) \leftarrow N(A) + 1$
  $Q(A) \leftarrow Q(A) + \frac{1}{N(A)}(R - Q(A))$

Por último, se incluye el algoritmo Softmax, que asigna probabilidades de selección proporcionales a los valores estimados de cada acción. Inicialmente, cada brazo se selecciona al menos una vez para evitar estimaciones vacías. La temperatura τ regula el grado de aleatoriedad: valores bajos favorecen políticas codiciosas, mientras que valores altos producen exploración casi uniforme. Esta estrategia permite al agente concentrarse en acciones prometedoras y evitar las de bajo rendimiento, como se detalla en el Algoritmo 3.

1.3.1. Tipos de brazos

Para evaluar la robustez y flexibilidad de los algoritmos seleccionados frente a distintos entornos estocásticos, se utilizaron tres tipos de brazos, cada uno con una distribución de recompensas específica: Normal, Binomial o Bernoulli. Esta variedad permite comparar el desempeño de los algoritmos en escenarios continuos y discretos, desde situaciones con incertidumbre hasta casos binarios simples.

En primer lugar, se implementaron brazos con distribución Normal $\mathcal{N}(\mu, \sigma^2)$, adecuados para modelar recompensas continuas. Estos generan recompensas reales que se distribuyen de forma simétrica alrededor de una media μ, con varianza σ^2 definida.

Posteriormente, se modelaron recompensas discretas mediante brazos con distribución Binomial $\mathcal{B}(n, p)$, que representa el número de éxitos en n ensayos independientes. De esta distribución se deriva la Bernoulli como caso particular con $n = 1$, el cual constituye el modelo más básico del bandido multibrazo, donde cada acción resulta en éxito o fracaso con una probabilidad fija.

1.4. Experimentos

La configuración experimental se ha diseñado para evaluar los algoritmos en un entorno controlado y reproducible. La implementación se ha realizado en Python, organizando los experimentos mediante *notebooks* ejecutados en Google Colab. Con el fin de garantizar la consistencia y repetibilidad de los resultados, se ha fijado una semilla común en los generadores de números aleatorios utilizados.

Para la obtención de las métricas se realizaron 500 ejecuciones independientes de cada experimento, con una duración de 1000 pasos por ejecución. Los resultados presentados corresponden al promedio de estas repeticiones, con el objetivo de reducir la variabilidad inherente a los procesos estocásticos y obtener una estimación más robusta del desempeño de los algoritmos.

El desempeño de los algoritmos se evalúa mediante tres métricas principales: la evolución de la recompensa promedio a lo largo del tiempo, el porcentaje de selecciones óptimas (que mide la rapidez con la que se identifica el mejor brazo) y el arrepentimiento acumulado (*regret*), definido como $R_T = \sum_{t=1}^{T}(q^* - r_t)$, donde q^* representa la recompensa del brazo óptimo y r_t la recompensa obtenida en cada paso. Esta última métrica permite cuantificar la pérdida asociada a la exploración de acciones subóptimas.

En cuanto a la configuración de los agentes, el algoritmo ϵ-Greedy se ha evaluado con $\epsilon \in \{0, 0{,}01, 0{,}1\}$. El caso $\epsilon = 0$ corresponde a una política completamente voraz tras una fase inicial de exploración, en la que cada acción es seleccionada al menos una vez para inicializar las estimaciones. Softmax se ha analizado con temperaturas $\tau \in \{0{,}1, 0{,}5, 1\}$, mientras que UCB1 se ha implementado siguiendo el enfoque basado en límites superiores de confianza (con $c = 1$). Todos los métodos se han evaluado sobre tres tipos de distribuciones de recompensa: Normal, Binomial y Bernoulli, fijando en el caso Binomial un valor de $n = 10$ para introducir variabilidad interna respecto al caso Bernoulli ($n = 1$).

Los resultados se presentan organizados por tipo de distribución. En cada subsección se analiza el comportamiento de los algoritmos ϵ-Greedy, UCB1 y Softmax bajo sus distintas configuraciones, comparando su velocidad de convergencia y estabilidad.

Para una visualización más detallada de las gráficas, estas se presentan divididas por algoritmo y tipo de distribución en los *notebooks* disponibles en el repositorio del proyecto [5].

1.4.1. Distribución normal

Recompensa promedio

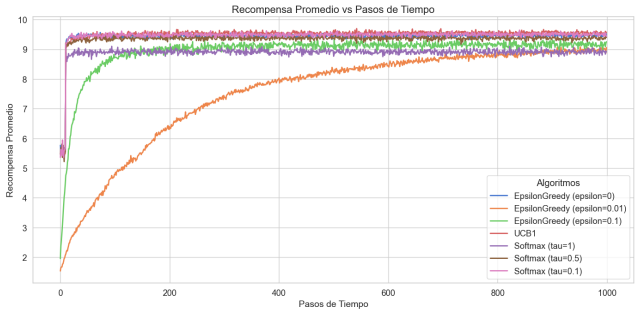
En la Figura 1.1a se observa que el rendimiento de los algoritmos depende principalmente de su capacidad para equilibrar exploración y explotación, lo que se refleja en diferencias tanto en la velocidad de convergencia como en la recompensa promedio final. En el caso de ϵ-Greedy, el valor de ϵ influye notablemente en el aprendizaje: valores bajos producen una convergencia más lenta, mientras que una mayor exploración inicial permite alcanzar antes recompensas elevadas. De forma puntual, la configuración voraz ($\epsilon = 0$) también alcanza buenos resultados, lo cual podría estar relacionado con la fase de exploración inicial forzada que garantiza estimaciones iniciales para todas las acciones. Por su parte, Softmax y UCB1 muestran un mejor comportamiento en etapas finales, destacando especialmente UCB1 por su rápida convergencia sin necesidad de ajuste manual de parámetros, mientras que valores altos de temperatura en Softmax generan un comportamiento excesivamente aleatorio que limita la recompensa obtenida.

Porcentaje de selecciones óptimas

En la Figura 1.1b se muestra la evolución del porcentaje de selecciones óptimas durante 1000 pasos de tiempo bajo una distribución normal de recompensas. UCB1 se comporta de manera más eficiente, convergiendo rápidamente hacia una tasa cercana al 100 % gracias a su manejo de la incertidumbre mediante límites superiores de confianza, lo que permite explorar brazos poco utilizados sin comprometer la explotación de los mejores [2]. Entre las estrategias ϵ-Greedy, $\epsilon = 0{,}1$ alcanza un rendimiento elevado tras un periodo inicial de aprendizaje, mientras que la versión puramente codiciosa ($\epsilon = 0$) asciende rápidamente al principio pero se estanca en un valor subóptimo, típico de la falta de exploración. Por su parte, Softmax depende de la temperatura: $\tau = 0{,}1$ permite una selección más precisa y competitiva frente a UCB1, mientras que $\tau = 1$ genera decisiones excesivamente estocásticas, dificultando consolidar el conocimiento sobre la distribución de recompensas.

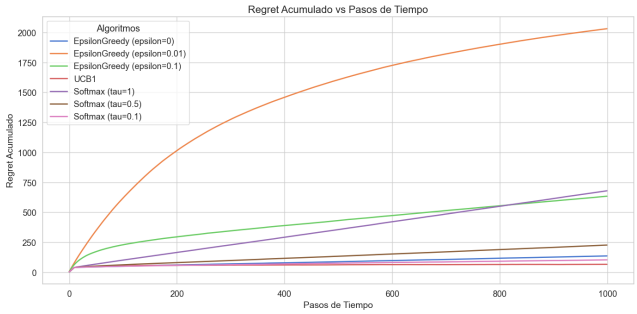
Arrepentimiento acumulado

En la Figura 1.1c se presenta el arrepentimiento acumulado de los algoritmos bajo una distribución normal de recompensas a lo largo de 1000 pasos. UCB1 muestra el desempeño más eficiente, con un arrepentimiento final mínimo gracias a su capacidad de reducir la incertidumbre sobre los brazos menos explorados y equilibrar exploración y explotación de manera logarítmica [2]. De manera similar, Softmax con temperatura baja ($\tau = 0{,}1$) mantiene niveles bajos de arrepentimiento, reflejando una convergencia precisa hacia el brazo óptimo. En contraste, ϵ-Greedy con $\epsilon = 0{,}1$ presenta un crecimiento constante por la exploración indefinida, mientras que $\epsilon = 0{,}01$ registra el mayor arrepentimiento debido a la insuficiente recolección de información inicial (baja probabilidad de exploración). La versión codiciosa ($\epsilon = 0$) muestra un arrepentimiento bajo en esta ejecución, pero corre el riesgo de quedar atrapada en brazos subóptimos por falta de exploración. Finalmente, Softmax con temperaturas elevadas ($\tau = 0{,}5$ y $\tau = 1$) experimenta un incremento del arrepentimiento, ya que la mayor estocasticidad impide consolidar el aprendizaje sobre la distribución normal [4].



(a) Recompensa promedio

(b) Selecciones óptimas



(c) Arrepentimiento acumulado

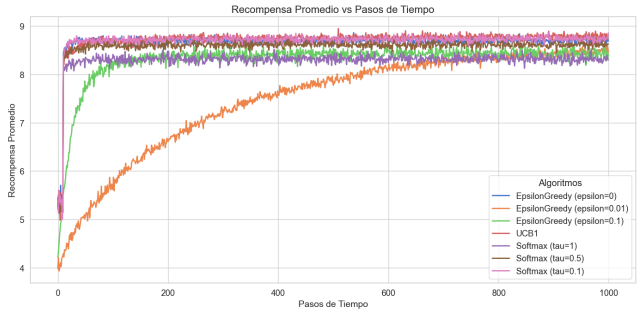
Figura 1.1: Resultados bajo una distribución normal de recompensas: evolución de la recompensa promedio, porcentaje de selecciones óptimas y arrepentimiento acumulado a lo largo del tiempo.

1.4.2. Distribución binomial

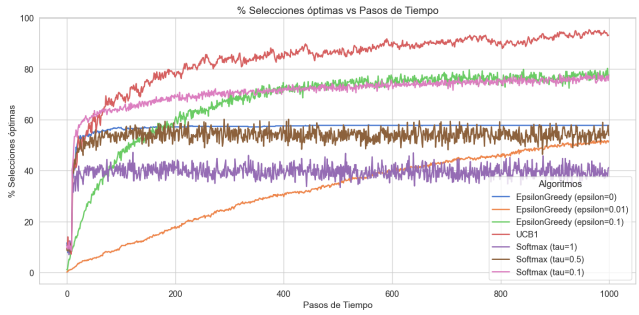
Recompensa promedio

En la Figura 1.2a se muestra la evolución de la recompensa promedio en un entorno con distribución binomial, donde la naturaleza acotada de las recompensas reduce la varianza de las estimaciones y favorece una convergencia más estable. En este escenario, UCB1 y Softmax con baja temperatura ($\tau = 0{,}1$) presentan el mejor desempeño, alcanzando rápidamente valores cercanos a la recompensa máxima y estabilizándose en menos de 50 pasos. En contraste, las variantes de ϵ-Greedy reflejan el compromiso entre exploración y explotación: $\epsilon = 0{,}01$ muestra un aprendizaje lento, mientras que $\epsilon = 0{,}1$ converge más rápido pero con una asíntota ligeramente inferior debido a la exploración constante. La es-

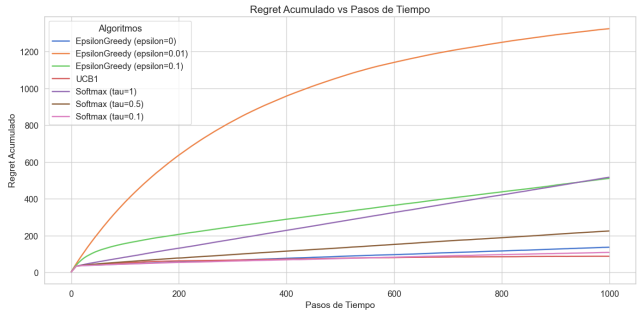
trategia voraz ($\epsilon = 0$) presenta un buen comportamiento en este experimento gracias al recorrido inicial de todos los brazos, lo que permite identificar rápidamente un brazo con alta recompensa estimada y estabilizar la recompensa promedio, aunque no necesariamente se trate del brazo óptimo. Finalmente, UCB1 destaca por su robustez teórica en este contexto de recompensas acotadas, mientras que Softmax con alta temperatura ($\tau = 1$) mantiene un comportamiento excesivamente estocástico que impide maximizar la recompensa promedio.



(a) Recompensa promedio



(b) Selecciones óptimas



(c) Arrepentimiento acumulado

Figura 1.2: Resultados bajo una distribución binomial de recompensas: evolución de la recompensa promedio, porcentaje de selecciones óptimas y arrepentimiento acumulado a lo largo del tiempo.

Porcentaje de selecciones óptimas

En la Figura 1.2b se compara la evolución del porcentaje de selecciones óptimas en el caso de recompensas binomiales. UCB1 muestra el desempeño más consistente, superando el 90 % hacia el final del experimento gracias a su manejo de la incertidumbre mediante límites superiores de confianza. Entre las estrategias ϵ-Greedy, $\epsilon = 0{,}1$ alcanza una convergencia estable cercana al 80 %, mientras que $\epsilon = 0{,}01$ progresa mucho más lento; la versión codiciosa ($\epsilon = 0$) asciende rápido al inicio pero se estanca en torno al 58 %, reflejando la falta de exploración y el riesgo de quedar atrapada en un brazo subóptimo. Para Softmax, una temperatura baja ($\tau = 0{,}1$) genera resultados competitivos concentrando la probabilidad en los brazos de mayor valor, mientras que temperaturas mayores ($\tau = 0{,}5$ o $\tau = 1$) inducen decisiones más estocásticas y un desempeño errático, evidenciando que una exploración demasiado uniforme dificulta consolidar el conocimiento sobre las probabilidades de éxito.

Arrepentimiento acumulado

En la Figura 1.2c se muestra el arrepentimiento acumulado de los algoritmos bajo una distribución binomial de recompensas a lo largo de 1000 pasos. De nuevo, UCB1 presenta el desempeño más robusto, con la curva de arrepentimiento más baja. Softmax con temperatura baja ($\tau = 0{,}1$) también logra resultados competitivos, concentrando la probabilidad de selección en los brazos con mayor valor estimado. En contraste, ϵ-Greedy con $\epsilon = 0{,}01$ registra el mayor arrepentimiento debido a una exploración insuficiente en las etapas tempranas, mientras que $\epsilon = 0{,}1$ identifica rápidamente el mejor brazo, aunque acumula arrepentimiento lineal por la exploración constante. La variante codiciosa ($\epsilon = 0$) muestra un arrepentimiento estable, pero corre el riesgo de quedarse bloqueada en brazos subóptimos si las recompensas iniciales no favorecen al brazo óptimo. Finalmente, Softmax con temperaturas elevadas ($\tau = 0{,}5$ y $\tau = 1$) incrementa el arrepentimiento linealmente, evidenciando de nuevo que una alta estocasticidad dificulta consolidar la política óptima.

1.4.3. Distribución bernoulli

Recompensa promedio

En la Figura 1.3a se muestra el rendimiento de los algoritmos en un entorno con recompensas de Bernoulli ($r_t \in \{0, 1\}$), caracterizado por su alta varianza. Los mejores comportamientos se observan en ϵ-Greedy con $\epsilon = 0{,}1$ y Softmax con $\tau = 0{,}1$, alcanzando recompensas promedio cercanas a 0.9 y 0.85, respectivamente. UCB1 logra estabilizarse alrededor de 0.8, mientras que la estrategia voraz ($\epsilon = 0$) también funciona bien en este experimento gracias al recorrido inicial de todos los brazos, aunque su trayectoria es más sensible a la varianza de las primeras muestras. Por el contrario, $\epsilon = 0{,}01$ resulta demasiado conservador y converge lentamente. Finalmente, Softmax con temperatura alta ($\tau = 1$) evidencia que una exploración excesiva dificulta la discriminación entre brazos, manteniendo la recompensa promedio en torno a 0.6, lo que subraya la importancia de un ajuste adecuado de τ.

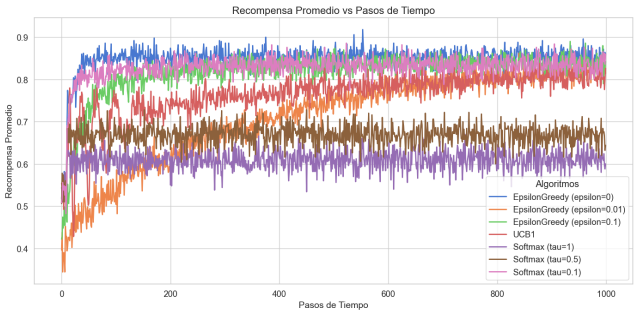
Porcentaje de selecciones óptimas

En la Figura 1.3b se compara el porcentaje de selecciones óptimas. La estrategia ϵ-Greedy con $\epsilon = 0{,}1$ muestra el desempeño más robusto, manteniéndose cerca del 60 % gracias a que la exploración constante permite refinar mejor las estimaciones de éxito de cada brazo. En contraste, la versión codiciosa ($\epsilon = 0$) se estanca alrededor del 48 %, reflejando el riesgo de quedar atrapada en brazos subóptimos al no explorar adecuadamente. UCB1 presenta una progresión más lenta y volátil, alcanzando apenas un 40 %, ya que los límites superiores de confianza tardan más en ajustarse con recompensas discretas. Para Softmax, únicamente una temperatura baja ($\tau = 0{,}1$)

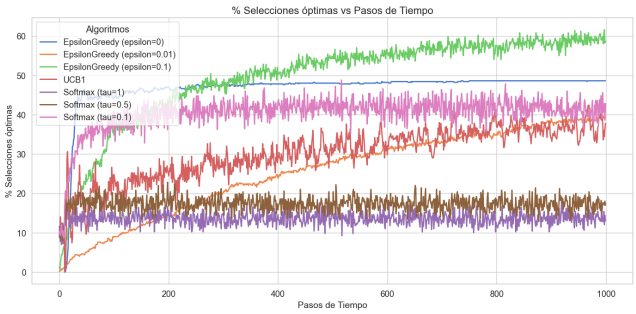
produce resultados competitivos, mientras que valores mayores incrementan demasiado la estocasticidad, dificultando la convergencia hacia el brazo óptimo.

Arrepentimiento acumulado

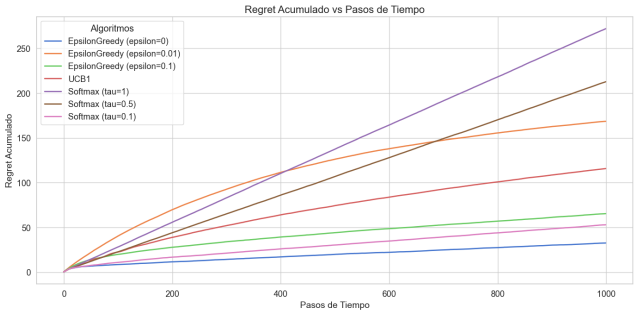
En la Figura 1.3c se muestra el arrepentimiento acumulado de los algoritmos. Softmax con temperatura alta ($\tau = 1$) presenta el crecimiento más pronunciado y lineal, debido a que la alta estocasticidad provoca una exploración excesiva de brazos subóptimos, dificultando aprovechar las acciones con mayor probabilidad de éxito. En contraste, UCB1 y ϵ-Greedy con $\epsilon = 0{,}1$ mantienen pendientes moderadas de arrepentimiento, destacando la capacidad de UCB1 para equilibrar exploración y explotación mediante límites de confianza superiores. La variante $\epsilon = 0{,}01$ acumula un arrepentimiento elevado, ya que su baja exploración retrasa la identificación del brazo óptimo. Finalmente, $\epsilon = 0$ y Softmax con $\tau = 0{,}1$ muestran los niveles más bajos de arrepentimiento; aunque $\epsilon = 0$ aprovecha bien los brazos explorados, carece de exploración activa y puede quedarse bloqueado en soluciones subóptimas si las recompensas iniciales no favorecen al brazo óptimo.



(a) Recompensa promedio



(b) Selecciones óptimas



(c) Arrepentimiento acumulado

Figura 1.3: Resultados bajo una distribución de bernoulli de recompensas: evolución de la recompensa promedio, porcentaje de selecciones óptimas y arrepentimiento acumulado a lo largo del tiempo.

1.5. Conclusiones

El análisis experimental mostró que los algoritmos UCB1 y Softmax con temperatura baja ($\tau = 0{,}1$) obtienen un desempeño consistente y eficiente en la mayoría de los entornos evaluados, alcanzando rápidamente altas recompensas promedio y manteniendo un bajo arrepentimiento acumulado. La estrategia ϵ-Greedy con $\epsilon = 0{,}1$ también resultó competitiva, especialmente en entornos binarios, mientras que variantes puramente codiciosas ($\epsilon = 0$) mostraron sensibilidad a las condiciones iniciales y riesgos de estancamiento en brazos subóptimos.

Por otro lado, la gráfica de arrepentimiento acumulado se considera la más importante. A diferencia de la recompensa promedio, que muestra el rendimiento en cada paso, el arrepentimiento refleja el coste total del aprendizaje, acumulando la diferencia entre la recompensa obtenida y la óptima teórica a lo largo del tiempo. Esta métrica permite identificar qué algoritmos aprenden de forma eficiente y cuáles se quedan atrapados en estrategias subóptimas, siendo un indicador habitual para evaluar la convergencia hacia el brazo óptimo.

Entre las limitaciones del estudio se encuentran los entornos estacionarios y simples, con parámetros fijos y sin ajuste adaptativo. Futuras mejoras podrían incluir entornos dinámicos, algoritmos con adaptación de parámetros y escenarios con más brazos o recompensas más complejas.

Este trabajo ayuda a entender cómo diferentes estrategias de exploración y explotación afectan el desempeño de los agentes en problemas de bandido de k-brazos. Los resultados muestran que métodos como UCB1 y Softmax, que balancean exploración y selección basada en valor, son efectivos y estables, proporcionando evidencia práctica de los principios fundamentales del aprendizaje por refuerzo en entornos controlados.

Como línea futura, se propone evaluar otros métodos como ϵ-decaimiento, UCB2 o Gradiente de Preferencias, entre otros. También resulta interesante analizar su desempeño en entornos no estacionarios, optimizar parámetros en tiempo real y extender los experimentos a escenarios más complejos.

Capítulo 2

Aprendizaje en Entornos Complejos

2.1. Introducción

En este capítulo se estudian y comparan distintos algoritmos de aprendizaje por refuerzo orientados al problema de control en dos escenarios con características diferentes. En particular, se analizan métodos tabulares aplicados a un entorno discreto y métodos basados en aproximaciones de función diseñados para trabajar en un entorno continuo. En este último caso, el espacio de estados se discretiza mediante agregación para poder aplicar SARSA semi-gradiente y Deep Q-Learning, permitiendo evaluar el comportamiento de cada enfoque bajo condiciones de complejidad distinta.

La relevancia de este estudio radica en analizar cómo los distintos algoritmos de control se comportan cuando el tamaño del espacio de estados o la naturaleza del entorno dificultan el aprendizaje. Los métodos tabulares resultan adecuados cuando es posible representar explícitamente las funciones de valor, mientras que las técnicas con aproximaciones permiten extender el aprendizaje a problemas donde la representación exacta no es viable.

El objetivo de este capítulo es implementar y comparar el desempeño de varios algoritmos de control en ambos tipos de entornos. Para ello, se estudian los métodos de Monte Carlo y de diferencias temporales en el caso discreto, así como SARSA semi-gradiente y Deep Q-Learning en el caso continuo discretizado, evaluando su estabilidad, velocidad de convergencia y calidad de las políticas aprendidas.

La organización del capítulo se divide en dos bloques principales. Primero se presentan los métodos tabulares, describiendo su fundamento teórico, los algoritmos utilizados y los experimentos realizados en el entorno discreto. Posteriormente, se introducen los métodos de control con aproximaciones de función sobre el entorno discretizado, siguiendo una estructura similar de desarrollo, implementación y evaluación experimental.

2.2. Métodos tabulares

2.2.1. Desarrollo

El problema abordado en este trabajo consiste en el aprendizaje de políticas de control óptimas en el entorno *Cliff Walking* de Gymnasium, el cual representa un escenario clásico de aprendizaje por refuerzo en espacios de estados discretos. Este problema consiste en guiar un agente desde un estado inicial hasta un objetivo final evitando caer en una zona penalizada que representa el acantilado [6].

En la literatura del aprendizaje por refuerzo, los problemas de control en entornos discretos han sido ampliamente estudiados mediante métodos basados en estimación de funciones de valor. Entre los enfoques más comunes se encuentran los métodos de Monte Carlo y las técnicas de diferencias temporales, los cuales permiten aprender políticas a partir de la experiencia generada mediante la interacción con el entorno.

El contexto de este trabajo se centra en el estudio de algoritmos de control capaces de equilibrar la exploración del entorno y la explotación de la información previamente aprendida. En particular, se analiza cómo diferentes estrategias de actualización de la función de valor influyen en la velocidad de convergencia y en la estabilidad del aprendizaje del agente en un entorno con penalizaciones asociadas a ciertos estados.

Los métodos utilizados para resolver el problema incluyen Monte Carlo *on-policy*, Monte Carlo *off-policy*, SARSA y Q-Learning. Estos algoritmos permiten comparar diferentes estrategias de aprendizaje, variando tanto el mecanismo de actualización de las estimaciones de valor como la forma en que se seleccionan las acciones durante el entrenamiento, con el objetivo de evaluar su rendimiento mediante métricas de recompensa acumulada y estabilidad del aprendizaje.

Este trabajo se diferencia de algunos enfoques previos en varios aspectos. No solo se implementan los algoritmos de control clásicos, sino que también se realiza una comparación sistemática de su comportamiento experimental. Todos los algoritmos se evalúan en el mismo entorno y bajo condiciones de entrenamiento homogéneas, lo que permite analizar de manera consistente las ventajas y limitaciones de cada método.

2.2.2. Algoritmos

En primer lugar, los algoritmos empleados en este apartado siguen el enfoque de control basado en estimación de funciones de valor. Los métodos de Monte Carlo se fundamentan en el aprendizaje a partir de episodios completos, donde las actualizaciones se realizan utilizando el retorno acumulado desde un estado-acción hasta el final del episodio. En el caso del método *on-policy*, la política de comportamiento coincide con la política que se está evaluando, mientras que en el enfoque *off-policy* se emplea muestreo por importancia para permitir el aprendizaje de una política objetivo distinta a la política exploratoria utilizada durante la generación de datos. Por su parte, los métodos de diferencias temporales, como SARSA y

Q-Learning, combinan ideas de Monte Carlo y programación dinámica, permitiendo actualizar las estimaciones de valor de manera incremental tras cada interacción con el entorno [1].

Los pseudocódigos presentados describen los pasos principales de cada algoritmo.

Algorithm 4 Algoritmo de Control Monte Carlo On-Policy (Every-Visit)

Inicializar: para cada estado $s \in \mathcal{S}$ y acción $a \in \mathcal{A}$:
  $Q(s,a) \leftarrow 0$
  $N(s,a) \leftarrow 0$
Parámetros: Probabilidad de exploración $\epsilon > 0$, factor de descuento $\gamma \in (0,1]$

Repetir para cada episodio:
  Actualizar ϵ si se utiliza decaimiento
  Generar un episodio siguiendo la política ϵ-greedy respecto a Q:
    $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  Repetir para cada paso del episodio $t = T-1, T-2, \ldots, 0$:
    $G \leftarrow \gamma G + R_{t+1}$
    $N(S_t, A_t) \leftarrow N(S_t, A_t) + 1$
    $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{N(S_t,A_t)} (G - Q(S_t, A_t))$

En el caso de Monte Carlo *on-policy* (Algoritmo 4), se inicializan las funciones de valor y los contadores de visitas, y posteriormente se generan episodios siguiendo una política ϵ-greedy. Las actualizaciones se realizan al finalizar cada episodio utilizando el retorno descontado.

Algorithm 5 Control Monte Carlo Off-Policy (Muestreo por Importancia)

Inicializar: para cada estado $s \in \mathcal{S}$ y acción $a \in \mathcal{A}(s)$:
  $Q(s,a) \leftarrow 0$
  $C(s,a) \leftarrow 0$
  $\pi(s) \leftarrow \arg\max_a Q(s,a)$
Parámetros: Factor de descuento $\gamma \in (0,1]$, límite de pasos $T_{\text{máx}}$

Repetir para cada episodio:
  $b \leftarrow$ política exploratoria
  Generar un episodio usando b (máximo $T_{\text{máx}}$ pasos):
    $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  $W \leftarrow 1$
  Para cada paso del episodio $t = T-1, T-2, \ldots, 0$:
    $G \leftarrow \gamma G + R_{t+1}$
    $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
    $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t,A_t)} (G - Q(S_t, A_t))$
    $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$
    Si $A_t \neq \pi(S_t)$ **entonces**
      Salir del bucle interno
    Fin Si
    $W \leftarrow W \frac{1}{b(A_t|S_t)}$

Para Monte Carlo *off-policy* (Algoritmo 5) se introduce un mecanismo de muestreo por importancia que permite corregir la diferencia entre la política exploratoria (ϵ-*Greedy* en nuestro caso) y la política objetivo (*greedy*). En particular, se emplea muestreo por importancia ponderado (*weighted importance sampling*), donde cada retorno observado se pondera mediante un peso que depende del cociente entre la probabilidad de la secuencia de acciones bajo la política objetivo y bajo la política de comportamiento. De este modo, las actualizaciones del valor de acción se ajustan para reflejar el comportamiento esperado si los episodios hubieran sido generados directamente por la política objetivo, reduciendo además la varianza de las estimaciones respecto al muestreo por importancia ordinario.

Algorithm 6 Algoritmo SARSA (Control TD On-Policy)

Inicializar: para cada estado $s \in \mathcal{S}$ y acción $a \in \mathcal{A}(s)$:
  $Q(s,a) \leftarrow 0$
Parámetros: Tasa de aprendizaje $\alpha \in (0,1]$, factor de descuento $\gamma \in (0,1]$, $\epsilon > 0$

Repetir para cada episodio:
  Actualizar ϵ si se utiliza decaimiento
  Inicializar estado S
  Elegir la acción A en S derivando la política de Q (ej. ϵ-greedy)
  Repetir para cada paso del episodio (hasta que S sea terminal):
    Ejecutar acción A, observar recompensa R y estado siguiente S'
    Elegir la acción A' en S' derivando la política de Q (ej. ϵ-greedy)
    $Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma Q(S', A') - Q(S,A)]$
    $S \leftarrow S'$
    $A \leftarrow A'$

En cuanto a SARSA, el algoritmo realiza actualizaciones utilizando la transición completa (S, A, R, S', A'), manteniendo la coherencia con la política seguida durante el entrenamiento (Algoritmo 6).

Algorithm 7 Algoritmo Q-Learning (Control TD Off-Policy)

Inicializar: para cada estado $s \in \mathcal{S}$ y acción $a \in \mathcal{A}(s)$:
  $Q(s,a) \leftarrow 0$
Parámetros: Tasa de aprendizaje $\alpha \in (0,1]$, factor de descuento $\gamma \in (0,1]$, $\epsilon > 0$

Repetir para cada episodio:
  Actualizar ϵ si se utiliza decaimiento
  Inicializar estado S
  Repetir para cada paso del episodio (hasta que S sea terminal):
    Elegir la acción A en S derivando la política de Q (ej. ϵ-greedy)
    Ejecutar acción A, observar recompensa R y estado siguiente S'
    $Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S,A)]$
    $S \leftarrow S'$

Finalmente, Q-Learning utiliza una estrategia *off-policy* que actualiza la función de valor empleando el máximo valor estimado del estado siguiente, lo que facilita la aproximación de la política óptima (Algoritmo 7).

La elección de estos algoritmos se debe a su aplicabilidad al problema de control en entornos discretos y a su capacidad para permitir la comparación entre estrategias *on-policy* y *off-policy*, así como entre métodos basados en episodios completos y métodos de actualización temporal. Monte Carlo resulta adecuado para problemas donde se puede esperar hasta el final del episodio para evaluar el retorno, mientras que SARSA y Q-Learning permiten un aprendizaje más eficiente mediante actualizaciones paso a paso. La comparación de estos métodos permite analizar diferencias en velocidad de convergencia, estabilidad del aprendizaje y calidad de las políticas obtenidas.

2.2.3. Experimentos

La implementación de los algoritmos se ha desarrollado mediante una arquitectura orientada a objetos, definiendo una clase abstracta general para el aprendizaje por refuerzo y clases específicas para cada método considerado. Esta organización permite modularizar la lógica de entrenamiento y facilitar la comparación entre los distintos enfoques. Cada algoritmo se ejecuta y entrena de manera independiente en un *notebook*, y posteriormente los resultados se integran en un *notebook* principal para realizar la comparación sistemática de desempeño.

Para la evaluación del desempeño se han utilizado dos métricas principales: la recompensa acumulada por episodio, que permite analizar la calidad de la política aprendida y su evolución durante el entrenamiento, y la longitud del episodio medida en número de pasos hasta alcanzar el estado terminal. Los algoritmos se han entrenado durante 5000 episodios, ya que experimentos preliminares con un mayor número de iteraciones no mostraron mejoras significativas en la recompensa media ni en la estabilidad de las políticas aprendidas. La representación gráfica de estas métricas permite estudiar la estabilidad, eficiencia y velocidad de convergencia de los algoritmos.

Como entorno de evaluación se ha seleccionado el problema *Cliff Walking*, un entorno discreto de control clásico que permite analizar el comportamiento de los agentes en una tarea de navegación con penalizaciones asociadas a ciertas regiones del espacio de estados. El entorno se ha configurado con renderizado en formato textual, mostrando la posición del agente mediante caracteres. Cada acción produce una recompensa de -1, mientras que las transiciones al estado del acantilado generan una penalización de -100, lo que incentiva evitar estas regiones del espacio de estados.

En cuanto a la política de selección de acciones, se ha empleado una estrategia ϵ-greedy que asigna una mayor probabilidad de selección a la acción con el mejor valor estimado según la función de valor Q, mientras que permite la exploración de otras acciones mediante una probabilidad uniforme controlada por el parámetro ϵ. Esta estrategia favorece el equilibrio entre exploración y explotación durante el entrenamiento de los algoritmos.

La Figura 2.1 presenta la evolución de la recompensa acumulada por episodio para los algoritmos Monte Carlo *on-policy*, Monte Carlo *off-policy*, SARSA y Q-Learning. Se ha empleado una política ϵ-greedy con $\epsilon = 0{,}3$ y decaimiento, favoreciendo una mayor exploración en las etapas iniciales y una explotación progresiva conforme avanza el entrenamiento. El factor de descuento se ha fijado en $\gamma = 0{,}95$, priorizando las recompensas futuras sin perder estabilidad. Para SARSA y Q-Learning se ha utilizado una tasa de aprendizaje $\alpha = 0{,}5$, permitiendo actualizaciones suficientemente significativas para acelerar el aprendizaje.

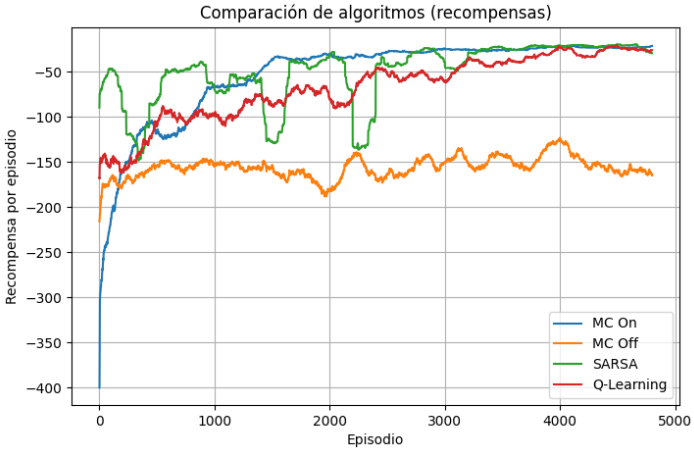


Figura 2.1: Recompensa promedio por episodio.

El comportamiento observado en el entorno *Cliff Walking* pone de manifiesto las diferencias entre los métodos de Diferencia Temporal (TD) y los métodos Monte Carlo (MC). Los algoritmos TD, como SARSA y Q-Learning, presentan una convergencia más rápida, ya que actualizan sus estimaciones tras cada transición mediante *bootstrapping*, sin necesidad de esperar al final del episodio. Ambos métodos alcanzan recompensas altas en menos episodios, aunque muestran diferencias en la forma en que gestionan el riesgo asociado al acantilado. En concreto, SARSA (*on-policy*) tiende a adoptar rutas más seguras, ya que su actualización considera la política realmente seguida, que incluye exploración con $\epsilon = 0{,}3$ y decaimiento. En cambio, Q-Learning (*off-policy*) aprende directamente hacia la política óptima estimada, lo que le permite bordear el acantilado y alcanzar la recompensa máxima posible.

Las oscilaciones observadas en la recompensa acumulada durante los primeros episodios en SARSA se explican por el carácter *on-policy* del algoritmo. Dado que la actualización de los valores de acción se realiza utilizando la acción efectivamente seleccionada por la política ϵ-greedy, el algoritmo incorpora el efecto de la exploración en el proceso de aprendizaje. En el entorno *Cliff Walking*, esto implica que trayectorias cercanas al precipicio pueden resultar especialmente sensibles a la exploración aleatoria, ya que una elección subóptima de acción puede provocar caídas con penalizaciones elevadas. Conforme el parámetro ϵ decae, la exploración disminuye, lo que permite que la política converja hacia rutas más seguras y que el comportamiento del agente se estabilice.

En cuanto a los métodos Monte Carlo, MC *on-policy* muestra un aprendizaje inicial lento debido a que las actualizaciones se realizan únicamente al finalizar cada episodio, lo que retrasa la mejora de la política en fases tempranas. No obstante, una vez estabilizado, su rendimiento se aproxima al de SARSA. Por su parte, MC *off-policy* muestra un comportamiento menos eficiente. Aunque se utiliza muestreo por importancia ponderado para reducir la varianza de las estimaciones,

la convergencia sigue siendo más lenta e inestable que en los métodos de diferencia temporal, especialmente en episodios con trayectorias largas.

En conjunto, los resultados indican que los métodos de Diferencia Temporal aprovechan mejor la información disponible en problemas de control discreto con penalización por paso, ofreciendo mayor eficiencia y estabilidad en comparación con los enfoques Monte Carlo, especialmente en su variante *off-policy*.

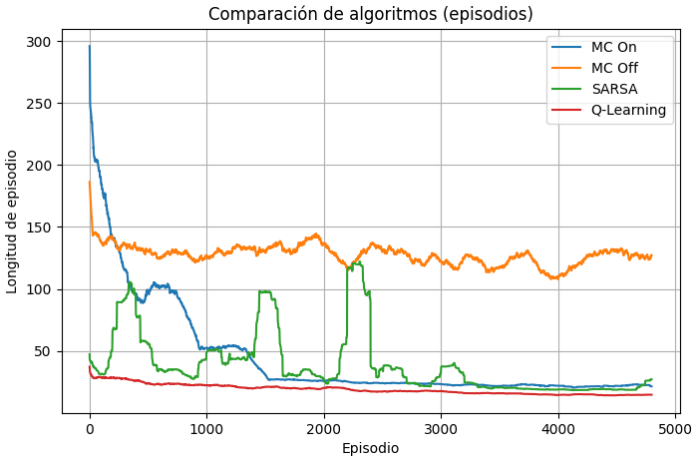


Figura 2.2: Longitud de episodio.

Por otro lado, el análisis de la longitud de los episodios permite evaluar la eficiencia de cada algoritmo para encontrar el camino más corto hacia la meta en *Cliff Walking* (Figura 2.2). Se observa que Q-Learning converge rápidamente a la longitud mínima posible y se mantiene estable, lo que refleja su capacidad para aprender la ruta óptima que bordea el acantilado. Este comportamiento es coherente con su carácter *off-policy*, ya que actualiza sus estimaciones hacia la mejor acción posible independientemente de la exploración realizada.

SARSA, por el contrario, converge a trayectorias ligeramente más largas y muestra mayor variabilidad. Al ser un método *on-policy*, incorpora el efecto de la exploración en su actualización, lo que le lleva a evitar zonas cercanas al acantilado y optar por caminos más seguros, aunque menos eficientes en número de pasos.

En cuanto a los métodos Monte Carlo, MC *on-policy* presenta episodios muy largos en las primeras fases debido a que solo actualiza al finalizar cada episodio, lo que retrasa la mejora de la política. Sin embargo, tras la fase inicial, logra reducir considerablemente la longitud de las trayectorias. MC *off-policy* mantiene una longitud elevada y comportamiento inestable durante gran parte del entrenamiento, afectado por la varianza del muestreo por importancia en episodios largos.

En conjunto, los resultados muestran que los métodos de Diferencia Temporal aprenden más rápido, destacando Q-Learning como el algoritmo que alcanza trayectorias más cortas y estables. SARSA tiende a priorizar rutas más seguras, mientras que Monte Carlo *on-policy* también logra políticas de buena calidad pero requiere más episodios para converger. Por su parte, Monte Carlo *off-policy* presenta mayor inestabilidad y menor eficiencia.

Finalmente, la Tabla 2.1 recoge los resultados obtenidos tras la ejecución de los experimentos. En este entorno, la recompensa máxima alcanzable es -13, correspondiente al camino más corto que bordea el acantilado.

Algoritmo	Recompensa máxima	Media últimos 100	Último episodio
MC On-Policy	-17.00	-20.64	-19.00
MC Off-Policy	-21.00	-166.76	-258.00
SARSA	-17.00	-31.99	-45.00
Q-Learning	-13.00	-27.67	-19.00

Tabla 2.1: Comparación de métricas de recompensa en el entorno Cliff Walking.

Se observa que Q-Learning es el único algoritmo que alcanza dicha recompensa máxima, lo que indica que es el método que logra aproximar con mayor precisión la política óptima del problema. Monte Carlo *on-policy* y SARSA alcanzan una recompensa máxima de -17, lo que sugiere que convergen hacia trayectorias algo más largas o conservadoras. Sin embargo, SARSA presenta una media en los últimos 100 episodios más baja que MC *on-policy*, indicando mayor variabilidad. Esto refleja cómo la actualización paso a paso de SARSA incorpora directamente la exploración durante el aprendizaje, favoreciendo rutas más seguras aunque menos eficientes.

Monte Carlo *off-policy* muestra el peor desempeño, tanto en la media de los últimos episodios como en la recompensa del último episodio, reflejando menor estabilidad en el aprendizaje. Aun usando muestreo por importancia ponderado para controlar la variabilidad, la diferencia entre la política exploratoria y la política objetivo puede generar actualizaciones menos precisas, especialmente en episodios largos, afectando la convergencia.

Dado que el objetivo del problema de control es maximizar la recompensa acumulada esperada, la política final seleccionada debería ser aquella que alcanza la mayor recompensa posible. En este caso, la política aprendida por Q-Learning sería la más adecuada, al ser la única que consigue el retorno máximo del entorno, incluso aunque su media en los últimos episodios no sea la más alta.

2.2.4. Conclusiones

En este capítulo se ha comparado el rendimiento de métodos Monte Carlo y de Diferencia Temporal en el entorno *Cliff Walking*. Los resultados muestran que los métodos de Diferencia Temporal, especialmente Q-Learning, convergen más rápido y alcanzan mejores recompensas, mientras que SARSA aprende políticas más conservadoras. Monte Carlo *on-policy* logra mejorar tras una fase inicial lenta, y Monte Carlo *off-policy* presenta mayor inestabilidad.

Como limitaciones, los resultados dependen de la elección de hiperparámetros y del número de episodios, además de haberse evaluado en un único entorno discreto. Futuras mejoras podrían incluir un análisis más sistemático de parámetros y la evaluación en otros entornos para reforzar la validez de las conclusiones.

Este trabajo resalta la importancia práctica de los métodos de Diferencia Temporal en tareas de control y permite compren-

der mejor las diferencias entre enfoques *on-policy* y *off-policy*, un aspecto central del aprendizaje por refuerzo moderno.

Como líneas futuras, se propone estudiar variantes más avanzadas de los algoritmos analizados, así como analizar estrategias de exploración alternativas y mejoras que aumenten la estabilidad y eficiencia del aprendizaje en problemas de mayor complejidad.

2.3. Control con aproximaciones

2.3.1. Desarrollo

En esta parte de la práctica se aborda el problema de control en aprendizaje por refuerzo en un entorno con espacio de estados continuo, modelado como un Proceso de Decisión de Markov $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$. El objetivo es aprender una política que maximice el retorno esperado acumulado. A diferencia de los métodos tabulares estudiados previamente, no es posible representar la función $Q(s, a)$ mediante una tabla, por lo que se recurre a aproximación de funciones para estimarla de forma paramétrica.

La aproximación de funciones en control se ha desarrollado como extensión natural de los métodos tabulares clásicos. Entre los enfoques más utilizados se encuentran los métodos con aproximadores lineales entrenados mediante gradiente estocástico, como SARSA semi-gradiente [1]. Más recientemente, el uso de redes neuronales profundas dio lugar al algoritmo Deep Q-Network (DQN), que incorpora técnicas como *experience replay* y redes objetivo para estabilizar el aprendizaje [8]. Variantes posteriores, como Double DQN, buscan reducir el sesgo por sobreestimación [9].

En los apartados anteriores se estudiaron primero el problema del bandido multi-brazo y posteriormente métodos tabulares de control en un espacio discreto. Estos enfoques permiten comprender los fundamentos del aprendizaje por refuerzo, pero presentan limitaciones cuando el espacio de estados es continuo o de gran dimensión. En este contexto, la aproximación de funciones resulta necesaria para permitir la generalización entre estados y la escalabilidad del algoritmo.

Por ello, se implementan dos enfoques de control con aproximación de funciones. Por un lado, SARSA semi-gradiente con aproximador lineal, donde la función $Q(s, a; \theta)$ se modela como una combinación lineal de características y los parámetros se actualizan mediante descenso de gradiente. Por otro lado, Deep Q-Learning, en el que una red neuronal profunda aproxima la función de valor y se entrena minimizando el error respecto al objetivo de Bellman, utilizando un *buffer* de experiencias y una red objetivo para mejorar la estabilidad.

El trabajo realiza una implementación y comparación experimental entre un aproximador lineal y uno profundo dentro del mismo entorno. La comparación permite analizar diferencias en estabilidad, convergencia y rendimiento final, evidenciando tanto las limitaciones del modelo lineal como la mayor capacidad de representación, y complejidad de entrenamiento, del enfoque basado en redes neuronales.

2.3.2. Algoritmos

En esta sección se describen los algoritmos utilizados para el control del agente mediante aproximación de funciones de valor, los cuales permiten el aprendizaje en espacios de estados donde no es factible una representación tabular de la función Q. Para ello, se emplean modelos paramétricos capaces de generalizar la información obtenida durante la interacción con el entorno. En particular, se analizan dos enfoques: SARSA semi-gradiente y Deep Q-Learning, que representan estrategias diferentes para la estimación y optimización de la función de valor-acción.

Algorithm 8 Sarsa Semi-gradiente Episódico para estimar $\hat{q} \approx q_*$

Entrada: una función de valor de acción diferenciable $\hat{q}(s, a, \boldsymbol{\theta})$ con pesos $\boldsymbol{\theta} \in \mathbb{R}^d$
Parámetros: tasa de aprendizaje $\alpha > 0$, descuento $\gamma \in (0, 1]$, exploración $\epsilon > 0$
Inicializar: vector de pesos $\boldsymbol{\theta}$ arbitrariamente (p. ej., $\boldsymbol{\theta} = \mathbf{0}$)

Repetir para cada episodio:
  Actualizar ϵ según el esquema de decaimiento lineal
  Inicializar estado S
  Elegir acción A en S usando política ϵ-greedy basada en $\hat{q}(S, \cdot, \boldsymbol{\theta})$
  Repetir para cada paso del episodio:
    Ejecutar acción A, observar recompensa R y estado siguiente S'
    Si S' es terminal **entonces**
      $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha[R - \hat{q}(S, A, \boldsymbol{\theta})]\nabla\hat{q}(S, A, \boldsymbol{\theta})$
      Salir al siguiente episodio
    Fin Si
    Elegir acción A' en S' usando política ϵ-greedy basada en $\hat{q}(S', \cdot, \boldsymbol{\theta})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha[R + \gamma\hat{q}(S', A', \boldsymbol{\theta}) - \hat{q}(S, A, \boldsymbol{\theta})]\nabla\hat{q}(S, A, \boldsymbol{\theta})$
    $S \leftarrow S'$
    $A \leftarrow A'$

El algoritmo SARSA semi-gradiente (Algoritmo 8) se basa en la actualización de los pesos de una función aproximadora diferenciable mediante el error temporal observado en cada transición del episodio. El agente selecciona acciones siguiendo una política ϵ-greedy con decaimiento progresivo, favoreciendo la exploración en las etapas iniciales del entrenamiento y la explotación del conocimiento adquirido posteriormente. La actualización de los parámetros se realiza en la dirección del gradiente de la función de valor-acción aproximada, minimizando la diferencia entre la estimación actual y el retorno esperado, lo que permite su aplicación en problemas de control con espacios de alta dimensionalidad.

Por otro lado, Deep Q-Learning (Algoritmo 9) utiliza una red neuronal profunda para aproximar la función de valor-acción, combinando el aprendizaje por refuerzo con técnicas de optimización basadas en descenso de gradiente. Este método emplea una memoria de repetición de experiencias que almacena transiciones del agente con el fin de romper la correlación entre muestras consecutivas y mejorar la estabilidad del entrenamiento. Asimismo, se introduce una red objetivo que se actualiza periódicamente para calcular el valor de re-

ferencia en la optimización, reduciendo la oscilación de las actualizaciones y mejorando la estabilidad del entrenamiento [8].

Algorithm 9 Algoritmo Deep Q-Learning (DQN con Repetición de Experiencia)

Inicializar:
  Memoria de repetición $\mathcal{D}$ con capacidad N
  Función de valor-acción Q con pesos aleatorios θ
  Función de valor-acción objetivo $\hat{Q}$ con pesos $\theta^- \leftarrow \theta$
  Contador global de pasos $C \leftarrow 0$
Parámetros: Tasa de aprendizaje α, factor de descuento $\gamma \in (0, 1]$, tamaño de lote B, frecuencia de actualización objetivo U, $\epsilon_{\text{inicial}}$, $\epsilon_{\text{mín}}$, $\epsilon_{\text{decaimiento}}$

Repetir para cada episodio:
  Inicializar estado S
  Repetir para cada paso del episodio (hasta que S sea terminal):
    Elegir la acción A en S derivando la política de Q (ej. ϵ-greedy)
    Ejecutar acción A, observar recompensa R, estado siguiente S' y terminación d
    Almacenar transición (S, A, R, S', d) en $\mathcal{D}$
    Si $|\mathcal{D}| \geq B$ **entonces**:
      Muestrear minilote aleatorio de transiciones $(S_j, A_j, R_j, S'_j, d_j)$ de $\mathcal{D}$
      Si el paso j es terminal (d_j es verdadero):
        $Y_j \leftarrow R_j$
      De lo contrario:
        $Y_j \leftarrow R_j + \gamma \max_{a'} \hat{Q}(S'_j, a'; \theta^-)$
      Realizar paso de descenso de gradiente en $(Y_j - Q(S_j, A_j; \theta))^2$ respecto a θ
    $S \leftarrow S'$
    $C \leftarrow C + 1$
    Si $C \pmod U == 0$ **entonces**:
      Actualizar la red objetivo: $\theta^- \leftarrow \theta$
  Actualizar ϵ: $\epsilon \leftarrow \max(\epsilon_{\text{mín}}, \epsilon \cdot \epsilon_{\text{decaimiento}})$

2.3.3. Experimentos

Para la implementación experimental, se ha estructurado el código de forma modular, definiendo una clase independiente para cada algoritmo de aprendizaje por refuerzo utilizado. Cada agente incorpora sus propios parámetros, política de interacción con el entorno y función de entrenamiento, lo que facilita la reproducibilidad de los experimentos y la comparación directa entre métodos. Cada algoritmo se ejecuta y entrena de manera independiente en un *notebook*, y posteriormente los resultados se integran en un *notebook* principal para realizar la comparación sistemática de desempeño.

En el caso de Deep Q-Learning, la implementación se ha dividido en tres componentes principales: el agente, la red neuronal y el *buffer* de repetición de experiencias. El *replay buffer* permite almacenar transiciones del entorno para posteriormente muestrearlas de forma aleatoria durante el entrenamiento, reduciendo la correlación temporal entre muestras y mejorando la estabilidad del aprendizaje, tal como se propone en la literatura [8].

La red neuronal utilizada como aproximador de la función Q es un perceptrón multicapa compuesto por una capa de entrada, dos capas ocultas de 128 neuronas cada una y una capa de salida con tantas unidades como acciones posibles. Entre las capas lineales se emplea la función de activación ReLU, que introduce no linealidad. Esta arquitectura permite mapear los estados discretizados del entorno a los valores $Q(s, a)$ asociados a cada acción, proporcionando capacidad de generalización dentro del espacio discretizado y facilitando el aprendizaje en un entorno originalmente continuo.

Para la evaluación del desempeño de los algoritmos se emplean dos métricas principales: la recompensa acumulada por episodio y la longitud del episodio. La recompensa permite medir la calidad de la política aprendida, mientras que la longitud del episodio refleja la eficiencia del agente para resolver la tarea. Estas métricas ya han sido utilizadas en la parte de métodos tabulares y permiten analizar la evolución del aprendizaje en función del número de episodios.

El entorno seleccionado para la experimentación es *Mountain Car*, un problema de control con espacio de estados continuo disponible en Gymnasium [7], en el que el objetivo es lograr que un vehículo alcance la cima de una colina mediante la aplicación de tres acciones discretas: acelerar hacia la izquierda, no acelerar y acelerar hacia la derecha. El estado del entorno está definido por la posición y la velocidad del vehículo. Aunque se aplica una discretización del espacio de estados, su tamaño hace poco práctico el uso de métodos tabulares, lo que motiva el empleo de aproximadores de funciones para permitir la generalización y el aprendizaje eficiente.

En este entorno, cada paso genera una recompensa de -1, y los episodios están limitados a un máximo de 200 pasos, lo que implica un retorno mínimo posible de -200 si no se alcanza el objetivo dentro del horizonte temporal establecido. Bajo esta estructura de recompensas, se considera que valores comprendidos aproximadamente entre -80 y -110 indican que el agente alcanza la meta en un número relativamente reducido de pasos.

Por lo tanto, para poder aplicar los algoritmos de control se ha realizado una discretización del espacio de estados mediante agregación de intervalos (*binning*). Se han probado diferentes configuraciones, incluyendo particiones de 20×20 y 30×30, obteniéndose mejores resultados con una discretización de 20×40, es decir, 20 intervalos para la posición y 40 para la velocidad. Esta elección se debe a que la dinámica del problema depende en mayor medida de la velocidad del agente, ya que permite acumular el impulso necesario para alcanzar la cima, por lo que una mayor resolución en esta variable favorece el aprendizaje de la política de control. Esto implica pasar de un número infinito de estados a uno finito; concretamente, 800 estados.

La política de selección de acciones empleada en ambos algoritmos es ϵ-greedy, que introduce un equilibrio entre exploración del espacio de estados y explotación de la información aprendida, empleado en algoritmos anteriores.

Para SARSA semi-gradiente se han utilizado los parámetros $\alpha = 0{,}05$, factor de descuento $\gamma = 0{,}99$ y $\epsilon = 0{,}3$ con decaimiento. El valor de la tasa de aprendizaje se ha seleccionado para permitir una convergencia relativamente rápida en un

aproximador lineal con estados discretizados mediante agregación, mientras que el factor de descuento favorece la consideración de recompensas futuras. El decaimiento de ϵ permite una exploración inicial más amplia y una posterior explotación de la política aprendida.

Para Deep Q-Learning se han empleado los parámetros $\alpha = 10^{-3}$, factor de descuento $\gamma = 0{,}99$, $\epsilon = 0{,}3$ con decaimiento, tamaño de batch de 64 y actualización de la red objetivo cada 50 pasos de entrenamiento. La tasa de aprendizaje se ha elegido para garantizar la estabilidad del entrenamiento de la red neuronal, mientras que el tamaño del batch permite un compromiso entre estabilidad estadística y coste computacional. La actualización periódica de la red objetivo contribuye a reducir la inestabilidad del entrenamiento de redes neuronales en aprendizaje por refuerzo [8].

En relación con la duración del entrenamiento, SARSA semi-gradiente se ejecutó durante 15000 episodios debido al aumento en la granularidad de la discretización del espacio de estados, lo que incrementa el número de combinaciones estado-acción que deben explorarse y ajustarse. Esta mayor complejidad requiere acumular más experiencia para estabilizar los pesos del aproximador lineal y lograr una generalización adecuada. Por su parte, Deep Q-Learning se entrenó durante 500 episodios, ya que el uso de una red neuronal como aproximador no lineal implica un mayor coste computacional por actualización. Además, en las pruebas iniciales se observó que el agente comenzaba a mejorar su rendimiento con un número relativamente reducido de episodios, por lo que se optó por mantener esta configuración al ofrecer un buen equilibrio entre eficiencia y desempeño.

En la Figura 2.3 se observa la evolución de la recompensa total obtenida por el agente utilizando el algoritmo SARSA semi-gradiente. Al inicio del entrenamiento, la recompensa se mantiene constante en -200, lo cual es coherente con la dinámica del entorno *Mountain Car*. A partir de aproximadamente el episodio 1500, se aprecia una mejora gradual en la recompensa media, lo que indica que el agente ha comenzado a generalizar el espacio de estados mediante la aproximación de funciones para encontrar la inercia necesaria. No obstante, la variabilidad en los valores individuales sugiere que la política sigue siendo significativamente exploratoria o que el aproximador lineal presenta dificultades para estabilizarse en ciertas regiones del espacio discretizado.

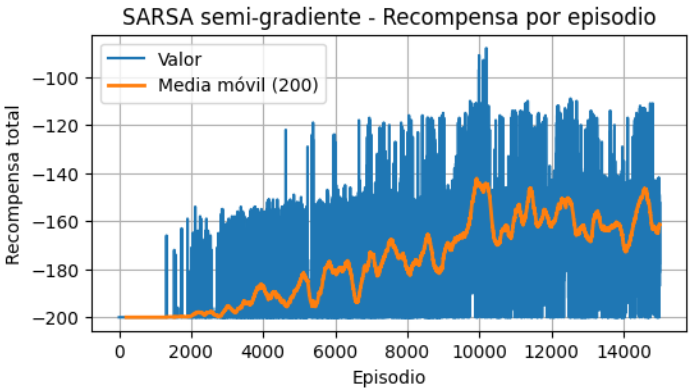


Figura 2.3: Recompensa por episodio (Sarsa semi-gradiente)

La Figura 2.4 detalla la cantidad de pasos necesarios para completar cada episodio, mostrando una tendencia inversamente proporcional a la recompensa. Se confirma que tras la fase inicial de estancamiento, el agente logra reducir el tiempo de ejecución, alcanzando mínimos por debajo de los 100 pasos en algunos casos. Sin embargo, la media móvil revela que el aprendizaje no es completamente lineal ni estable, pues existen repuntes hacia los 200 pasos incluso en etapas avanzadas. Este comportamiento es típico en métodos *on-policy* como SARSA, donde la actualización de los pesos depende de la acción siguiente tomada por la política actual, la cual incluye un factor de exploración que puede llevar al agente a estados subóptimos de manera recurrente.

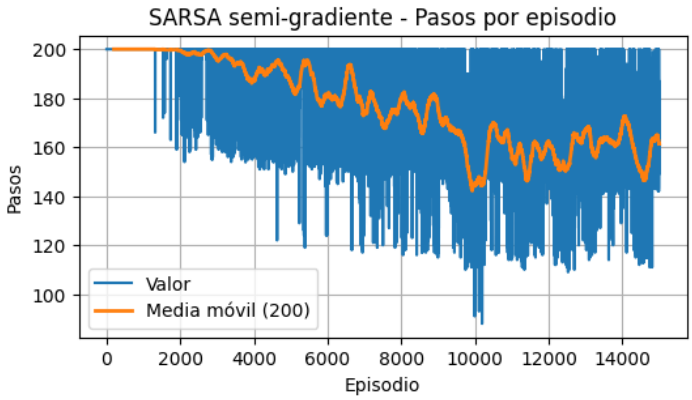


Figura 2.4: Pasos por episodio (Sarsa semi-gradiente)

En la Figura 2.5, que representa la recompensa total por episodio, se observa un periodo inicial de estancamiento que se extiende hasta aproximadamente el episodio 210. Durante esta fase, la recompensa se mantiene constante en -200, lo que indica que el agente no logra alcanzar la meta antes de agotar el límite de pasos permitido por el entorno. Este comportamiento es característico de problemas con recompensas dispersas, donde el agente requiere una exploración inicial aleatoria exitosa para comenzar a nutrir el *buffer* de repetición de experiencia con transiciones positivas. A partir de dicho punto, se aprecia una tendencia ascendente en la media móvil, alcanzando valores cercanos a -140 hacia el final del entrenamiento. La variabilidad observada en los valores individuales es propia de la naturaleza *off-policy* del algoritmo y del uso de una política ϵ-greedy, la cual introduce ruido deliberado para continuar explorando el espacio de estados incluso cuando la política ya muestra signos de optimización.

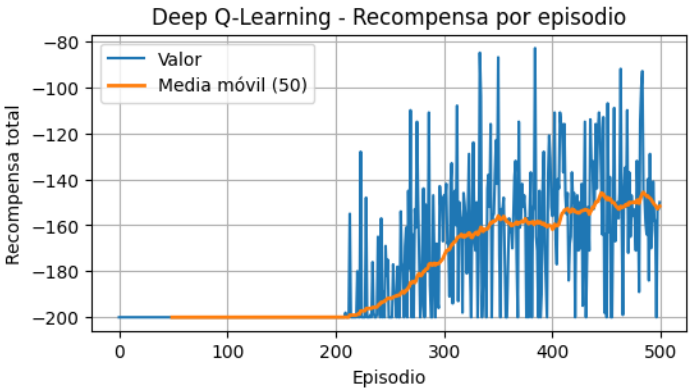


Figura 2.5: Recompensa por episodio (Deep Q-Learning)

La Figura 2.6 detalla la evolución de los pasos por episodio y confirma la mejora en la eficiencia de la política aprendida. De manera inversamente proporcional a la recompensa, el número de pasos comienza a descender una vez superado el umbral del episodio 210, llegando a registrar episodios exitosos con menos de 100 pasos. El hecho de que la media móvil se estabilice en torno a los 150 pasos sugiere que la red neuronal ha logrado generalizar de forma adecuada la relación entre la posición y la velocidad para superar la pendiente. La estabilidad relativa en la fase final del entrenamiento se ve favorecida por la actualización periódica de la red de destino cada 50 pasos, lo cual mitiga las oscilaciones en las estimaciones de los valores Q y permite una convergencia más robusta hacia una política prácticamente óptima.

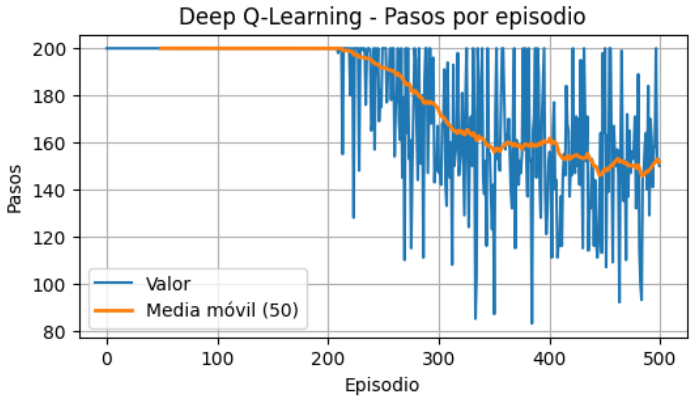


Figura 2.6: Pasos por episodio (Deep Q-Learning)

Finalmente, la Tabla 2.2 presenta los resultados obtenidos en *Mountain Car*. Como se ha mencionado anteriormente, podemos considerar que valores comprendidos entre -80 y -110 indican que el agente logra alcanzar la meta en un número relativamente reducido de pasos.

Algoritmo	Recompensa máxima	Media últimos 100	Último episodio
SARSA semi-gradiente	-88.00	-161.49	-181.00
Deep Q-Learning	-83.00	-149.80	-150.00

Tabla 2.2: Comparación de métricas de recompensa en el entorno Mountain Car.

Los resultados muestran que Deep Q-Learning obtiene el mejor desempeño entre los algoritmos evaluados, alcanzando una recompensa máxima de -83, valor que se encuentra dentro del rango considerado óptimo para el problema. Además, presenta una mejor media en los últimos 100 episodios y una mayor estabilidad en el aprendizaje en comparación con SARSA semi-gradiente. Esto sugiere que la capacidad de representación de las redes neuronales profundas permite aproximar de manera más precisa la función de valor en este entorno.

SARSA semi-gradiente, por su parte, obtiene un rendimiento ligeramente inferior (aunque aún dentro del rango considerado óptimo), con una recompensa máxima de -88 y una mayor variación en la evolución de la recompensa acumulada. Esto se debe a que los modelos lineales o semiparamétricos tienen menos capacidad para capturar con precisión cómo las acciones del agente influyen en los estados futuros y las recompensas, es decir, la dinámica del entorno. En comparación, los modelos profundos pueden representar estas relaciones más complejas, lo que les permite generalizar mejor y alcanzar un desempeño superior.

En problemas de aprendizaje por refuerzo, la política final suele elegirse como aquella que maximiza la recompensa esperada. En este caso, Deep Q-Learning sería la opción preferible, ya que aproxima mejor la política óptima del entorno *Mountain Car*.

2.3.4. Conclusiones

En esta sección se han comparado dos enfoques de control con aproximación de funciones aplicados al entorno *Mountain Car*. Los resultados muestran que tanto SARSA semi-gradiente como Deep Q-Learning permiten mejorar progresivamente la política de control del agente a lo largo del entrenamiento. En particular, SARSA semi-gradiente logra un aumento moderado de la recompensa respecto al rendimiento inicial, aunque presenta mayor variabilidad durante el aprendizaje, mientras que Deep Q-Learning alcanza valores de recompensa más favorables y una mayor reducción en la longitud de los episodios hacia las fases finales del entrenamiento.

A pesar de los resultados obtenidos, el estudio presenta algunas limitaciones. La principal se relaciona con la discretización del espacio de estados, que introduce una aproximación del problema continuo original y puede limitar la capacidad de generalización del agente. Además, en todos los algoritmos se han fijado determinados hiperparámetros durante el entrenamiento. Un análisis más exhaustivo podría incluir una búsqueda sistemática de estos hiperparámetros o el promedio de los resultados obtenidos con múltiples semillas aleatorias, con el fin de evaluar la robustez y consistencia de las políticas aprendidas.

Este trabajo demuestra la importancia del aprendizaje por refuerzo como herramienta para resolver problemas de control en entornos dinámicos. En particular, los métodos basados en aproximación de funciones permiten extender las ideas de los algoritmos tabulares clásicos a espacios de estados continuos.

Como líneas futuras de trabajo, sería interesante explorar el uso de aproximadores no lineales alternativos, como redes neuronales con arquitecturas más profundas o métodos actor-crítico que permitan manejar de forma natural espacios de acción continuos. También podría considerarse la evaluación del desempeño del agente bajo diferentes esquemas de discretización del entorno y el análisis del impacto de los hiperparámetros sobre la convergencia del algoritmo.

2.4. Conclusiones generales

En este segundo capítulo se han analizado distintos enfoques de aprendizaje por refuerzo aplicados a problemas de control en entornos discretos y continuos. En el caso de los métodos tabulares, los resultados en *Cliff Walking* evidencian que los algoritmos de Diferencia Temporal presentan, en general, una convergencia más rápida y un mejor rendimiento final que los métodos Monte Carlo. Dentro de los métodos de Diferencia Temporal, Q-Learning destaca por alcanzar mayores recompensas, mientras que SARSA tiende a aprender políticas más conservadoras debido a su naturaleza *on-policy*. Por su parte, en el entorno *Mountain Car*, los métodos con aproximación de funciones han demostrado ser esenciales para abordar espacios de estados originalmente continuos, que

se han discretizado mediante un esquema de *bins*. En este escenario, Deep Q-Learning alcanza un desempeño superior al de SARSA semi-gradiente, tanto en términos de recompensa acumulada como de reducción de pasos por episodio.

En conjunto, los resultados ponen de manifiesto dos ideas clave. En primer lugar, que los métodos *off-policy* pueden ofrecer un rendimiento superior en algunos casos, como se observa con Q-Learning en el entorno *Cliff Walking*, mientras que otras implementaciones, como Monte Carlo *off-policy*, muestran mayor inestabilidad. En segundo lugar, la aproximación de funciones permite aplicar algoritmos como SARSA semi-gradiente y Deep Q-Learning sobre el entorno continuo discretizado mediante *bins*, facilitando la generalización a partir de la experiencia y la mejora progresiva de la política de control.

No obstante, las conclusiones obtenidas deben interpretarse teniendo en cuenta ciertas limitaciones, como la dependencia de los resultados respecto a la elección de hiperparámetros, el número de episodios de entrenamiento y la evaluación en un conjunto reducido de entornos.

En definitiva, este estudio proporciona una visión progresiva que conecta los fundamentos tabulares del aprendizaje por refuerzo con sus extensiones basadas en aproximación, mostrando de forma práctica cómo evolucionan las técnicas a medida que aumenta la complejidad del problema.

Bibliografía

[1] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press. https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf

[2] Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). *Finite-time analysis of the multiarmed bandit problem*. Machine Learning, 47(2–3), 235–256. https://homes.di.unimi.it/~cesabian/Pubblicazioni/ml-02.pdf

[3] Lattimore, T., & Szepesvári, C. (2020). *Bandit algorithms*. Cambridge University Press. https://tor-lattimore.com/downloads/book/book.pdf

[4] Vermorel, J., & Mohri, M. (2005). *Multi-armed bandit algorithms and empirical evaluation*. European Conference on Machine Learning (ECML). https://cs.nyu.edu/~mohri/pub/bandit.pdf

[5] Ouardi, M., Montesinos, J., & Oliva, M. (2026). *EML_MontesinosOlivaOuardi* Repositorio GitHub. https://github.com/mariiam36/EML_MontesinosOlivaOuardi

[6] Farama Foundation. (2023). *Cliff Walking*. Gymnasium Documentation. https://gymnasium.farama.org/environments/toy_text/cliff_walking/

[7] Farama Foundation. (2023). *Mountain Car*. Gymnasium Documentation. https://gymnasium.farama.org/environments/classic_control/mountain_car/

[8] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., ... & Hassabis, D. (2015). *Human-level control through deep reinforcement learning*. Nature, 518, 529–533. https://www.nature.com/articles/nature14236

[9] Van Hasselt, H., Guez, A., & Silver, D. (2016). *Deep reinforcement learning with double Q-learning*. arXiv. https://arxiv.org/abs/1509.06461